EDGE AI · GOOGLE AI EDGE

Despliegue de aprendizaje automático en sistemas embebidos: el caso de LiteRT

Presentado por

Eduardo Tencio Solano

Mini-Taller de Sistemas Embebidos

Instituto Tecnológico de Costa Rica · Escuela de Ingeniería Electrónica



AGENDA

Lo que veremos hoy

De la idea general a los números reales

1	**El desafío del borde**	Por qué un buen modelo falla fuera del servidor
2	**¿Qué hace y cómo trabaja?**	El camino de un modelo y los motores que lo ejecutan
3	**Compresión y modelos**	Cuantización y dónde conseguir modelos listos
4	**Evaluación**	Qué esperar según datos reales
5	**Conclusiones y demo**	Resumen, preguntas y práctica en Linux

TAKEAWAY **Qué es, qué hace y cómo trabaja LiteRT, en lenguaje sencillo.**

Del laboratorio al borde

Un modelo excelente en el servidor puede fallar en el dispositivo

EJEMPLO ILUSTRATIVO

Un modelo de visión entrenado en un servidor con GPU llega a 98 % de precisión y responde al instante. Luego se despliega en una computadora embebida de una línea de producción.

La solución: LiteRT, de Google AI Edge

Memoria
La RAM del equipo no alcanza (error OOM)

Latencia
Segundos por cuadro, no milisegundos

Energía
El consumo limita al dispositivo

TAKEAWAY **La brecha entre la nube y el hardware del borde es el obstáculo real.**

¿Por qué procesar en el borde?

Llevar la inteligencia a donde nacen los datos

Respuesta rápida
Sin ir y volver a un servidor, la respuesta es más ágil.

Privacidad
Los datos sensibles se quedan en el dispositivo.

Menos tráfico de red
No hace falta enviar imágenes o audio a la nube.

Resiliencia
El equipo sigue funcionando aunque la red falle.

TAKEAWAY **Procesar en el dispositivo da rapidez, privacidad y resiliencia.**

¿Qué es LiteRT?

Lite Runtime: el motor de Google AI Edge que ejecuta IA en el dispositivo

1 · Convertir

Desde PyTorch, TensorFlow, JAX o Keras al formato .tflite.

2 · Comprimir

Cuantización: modelos más pequeños y livianos.

3 · Ejecutar

En CPU, GPU o NPU, según el hardware disponible.

Su foco es ejecutar modelos ya entrenados; el entrenamiento se hace en PyTorch, TensorFlow o JAX.

TAKEAWAY **LiteRT lleva modelos ya entrenados al dispositivo para que corran allí.**

De TensorFlow Lite a LiteRT

Un cambio de nombre que amplió el alcance

Cuándo

4 de septiembre de 2024: TensorFlow Lite pasa a llamarse LiteRT.

Por qué

Dejar de depender de TensorFlow y aceptar modelos de PyTorch, JAX y Keras.

Qué no cambia

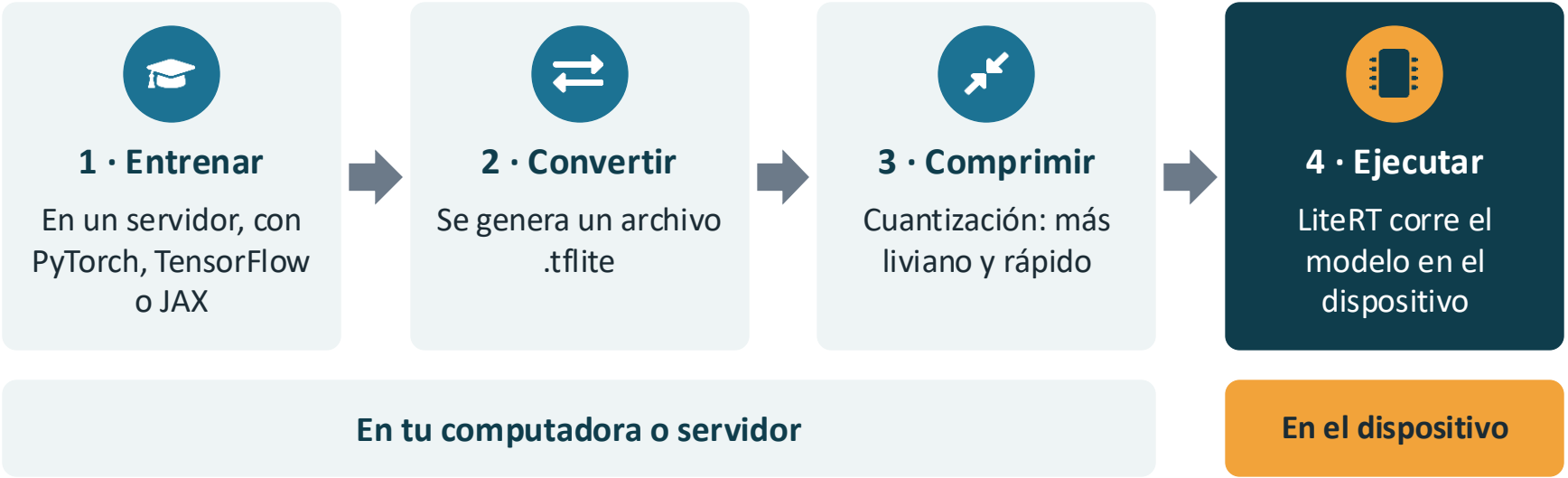
El formato .tflite y las apps ya desplegadas siguen funcionando igual.

Hoy: LiteRT 2.x trae la API CompiledModel; el paquete clásico queda en mantenimiento.

TAKEAWAY **Cambió el nombre y creció el alcance, sin romper lo ya desplegado.**

El camino de un modelo

Del entrenamiento al dispositivo, en cuatro pasos



TAKEAWAY **Se entrena una vez en la nube y se ejecuta en muchos dispositivos.**

¿Dónde se usa LiteRT?

De apps que usamos a diario hasta sensores y robots

250K+

aplicaciones en producción

Presente en apps de Google como YouTube, Photos, Gmail, Maps, Chrome y Meet.

Android · iOS · Web · Linux · macOS · Windows · IoT



Visión
Clasificación, detección y segmentación



Voz y audio
Reconocimiento de voz en tiempo real



Lenguaje
Asistentes y modelos como Gemma



IoT y robótica
Anomalías y fusión de sensores

TAKEAWAY **Ya está en productos de uso diario, del celular a los sensores.**

Cómo ejecuta un modelo

Tres pasos simples: cargar, elegir dónde y ejecutar

1 · Cargar
Se abre el archivo .tflite del modelo

2 · Elegir
Se indica si corre en CPU, GPU o NPU

3 · Ejecutar
Se entregan los datos y se leen los resultados

CompiledModel · API moderna
Prepara el modelo para el hardware elegido y maneja las entradas y salidas. Es la opción recomendada.

Interpreter · API clásica
Heredada de TensorFlow Lite. Requiere más configuración manual y se mantiene por compatibilidad.

TAKEAWAY **La API moderna (CompiledModel) prepara el modelo para el hardware elegido.**

CPU, GPU y NPU

Tres motores para el mismo modelo

CPU

El todoterreno

Corre en cualquier dispositivo. Es la opción por defecto y la más portable.

GPU

El especialista en paralelo

Hace muchas operaciones a la vez; ayuda en visión y modelos grandes.

NPU

El especialista en IA

Chip creado para redes neuronales: rápido y eficiente, pero depende del fabricante.

Plan B automático: si el acelerador no soporta una operación, LiteRT la pasa a otro (fallback).

TAKEAWAY **Tú eliges el acelerador; LiteRT se encarga de que el modelo siga funcionando.**

Aceleradores por plataforma

Qué puede aprovechar LiteRT en cada sistema

Plataforma	CPU	GPU	NPU
Android	Sí	OpenCL, OpenGL	Qualcomm, MediaTek, Google Tensor, Samsung
iOS / macOS	Sí	Metal, WebGPU	En desarrollo
Web	Sí	WebGPU	En desarrollo
Linux	Sí	WebGPU, OpenCL	Según fabricante*
Windows	Sí	WebGPU, OpenCL	Intel, Qualcomm
IoT	Sí	WebGPU	Qualcomm

TAKEAWAY **El soporte cambia con la plataforma y el fabricante: conviene verificarlo.**

Cuantización: la idea

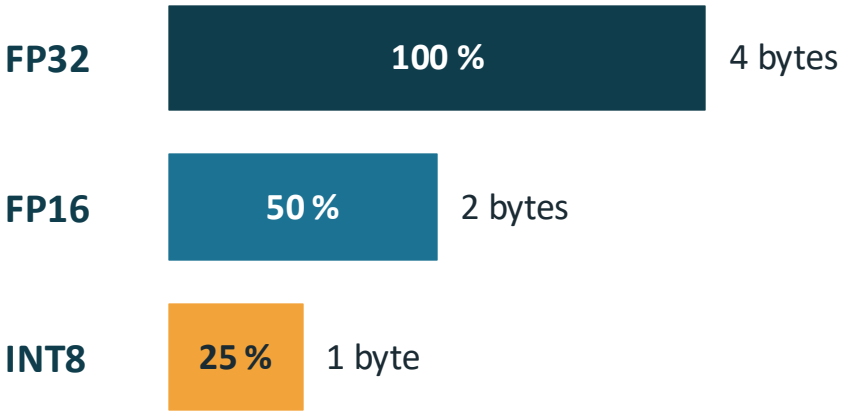
Guardar los números con menos detalle, sin reconstruir el modelo

Como comprimir una foto

Se guarda con menos detalle: pesa mucho menos y casi no se nota.

Para dimensionar: cada mil millones de parámetros en FP32 ocupan unos 4 GB.

Tamaño relativo por parámetro



TAKEAWAY **INT8 reduce el tamaño del archivo a una cuarta parte.**

Qué se gana y qué se pierde

La cuantización es un intercambio, no magia

Se gana

- Archivos hasta 4 veces más pequeños
- Menos datos viajando entre memoria y procesador
- Más velocidad en CPU y aceleradores enteros
- Menos consumo de energía

Hay que cuidar

- Una pequeña pérdida de precisión: se mide, no se supone
- En GPU suele preferirse FP16; INT8 brilla en CPU y NPU
- El ahorro es de almacenamiento: la RAM en ejecución baja menos

TAKEAWAY **Se gana tamaño y velocidad a cambio de una pérdida de precisión que hay que medir.**

Estrategias: PTQ vs QAT

Comprimir después de entrenar o entrenar pensando en la compresión

PTQ · después de entrenar

No requiere reentrenar. Es rápida, simple y económica.

QAT · durante el entrenamiento

El modelo aprende a convivir con la compresión. Más fiel, pero más trabajo.

1 · Probar PTQ
Es lo más rápido

→

2 · Medir
Comparar con el original

→

3 · Si cae mucho
QAT o precisión mixta

TAKEAWAY **Primero PTQ por rapidez; QAT solo si la pérdida de precisión es inaceptable.**

Cómo conseguir un modelo

Dos caminos: convertir el tuyo o descargar uno ya listo

Convertir el tuyo

- PyTorch: litert-torch (antes ai-edge-torch) lo lleva a .tflite
- TensorFlow, Keras y JAX: sus conversores habituales

Descargar uno listo

- LiteRT Community en Hugging Face: modelos de visión, voz y lenguaje
- Es comunitario: revisa licencia y origen

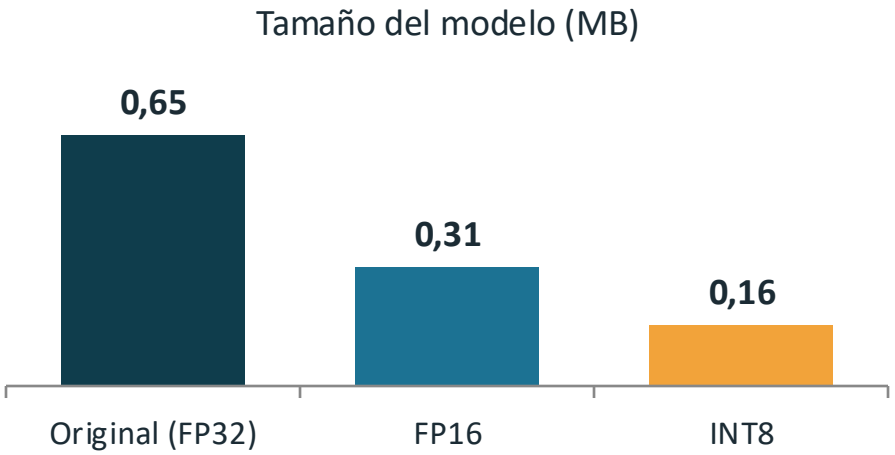
.tflite visión y audio, con LiteRT

.litertlm lenguaje, con LiteRT-LM (sobre LiteRT)

TAKEAWAY **Se puede convertir un modelo propio o descargar uno ya convertido.**

Datos reales de compresión

Un ejemplo medido en la literatura



≈ ½ del tamaño con FP16, con precisión igual o mayor

≈ ¼ del tamaño con INT8, con una caída de 0,75 a 2,72 puntos de precisión

Modelo GBRAS-Net de esteganálisis (Ferreira et al., 2025). INT8 = cuantización de rango dinámico. Los resultados varían con el modelo y el hardware.

TAKEAWAY **Los números confirman la teoría, pero siempre hay que medir con tu propio modelo.**

Portabilidad frente a rendimiento

Un mismo modelo en muchos equipos, o el máximo en uno solo

Portabilidad

Un mismo modelo y formato corre en Android, iOS, Web, Linux, Windows e IoT.

Rendimiento máximo

Un motor dedicado, como TensorRT en NVIDIA, puede rendir más en su propio hardware.

Qué hacer

Medir latencia y precisión en tu hardware antes de decidir.

Evidencia: Sobhani et al. (2025) comparó Raspberry Pi, Intel NCS, Google Coral y Jetson Nano; Jetson con TensorRT fue la configuración más eficiente. Su prueba usó la API clásica (Interpreter).

TAKEAWAY **Portabilidad y rendimiento máximo son un intercambio: mide para decidir.**

Conclusiones

Las cuatro ideas que nos llevamos

1

Qué es
El motor que convierte, comprime y ejecuta IA directamente en el dispositivo.

2

Cómo trabaja
Un archivo .tflite, la API CompiledModel y aceleración en CPU, GPU o NPU.

3

Compresión
La cuantización reduce el archivo hasta una cuarta parte; hay que validar la precisión.

4

Compromiso
Portabilidad frente al máximo rendimiento de un motor dedicado.

TAKEAWAY **LiteRT cierra la brecha del borde: convertir, comprimir y ejecutar con portabilidad.**

BLOQUE 5 | CONCLUSIONES

¡Muchas gracias por su atención!

Preguntas y comentarios sobre la parte teórica

SIGUIENTE PASO

Demostración práctica en Linux

TAKEAWAY **Pasemos de la teoría a la práctica: demostración en consola Linux.**

Fuentes consultadas

Documentación oficial y estudios usados en esta presentación

[1] Google AI Edge Team (2024). TensorFlow Lite is now LiteRT. Google for Developers Blog. developers.googleblog.com/en/tensorflow-lite-is-now-litert

[2] Ferreira, G., Marinho, M., Severo, V. y Madeiro, F. (2025). Optimization strategies applied to deep learning models for image steganalysis. Applied Sciences, 15(9), 4632. mdpi.com/2076-3417/15/9/4632

[3] Sobhani, G., Ifath, M., Sharma, T. y Haque, I. (2025). On the sustainability of AI inferences in the edge. arXiv:2507.23093. arxiv.org/abs/2507.23093

[4] Lopez Mendez, R. (2023). Neural network model quantization on mobile. Arm Community. developer.arm.com/community/arm-community-blogs

[5] Chadha, A. Model compression for on-device AI. Aman's AI Journal. aman.ai/primers/ai/model-compression

[6] Google. LiteRT: high-performance on-device ML framework. Google AI Edge. developers.google.com/edge/litert

[7] Google AI Edge. litert-samples (samples/litert). GitHub. github.com/google-ai-edge/litert-samples

[8] LiteRT Community. Hugging Face. huggingface.co/litert-community

[9] Google AI Edge. litert-torch. GitHub. github.com/google-ai-edge/litert-torch

[10] Google AI Edge. LiteRT. GitHub. github.com/google-ai-edge/litert

[11] Google. Getting started with LiteRT. Google AI Edge. developers.google.com/edge/litert/overview